



nextwork.org

Give Your Assistant a Memory



tahirgroot@gmail.com

<https://medium.com/@tahirbalarabe2/how-to-m...>



Introducing Today's Project!

In this project, I'm going to give my OpenClaw assistant a persistent memory and a distinct personality. Right now, most chat apps forget everything when I start a new conversation. But here, I'll make sure my assistant actually remembers me across sessions. Specifically, I'll create two key files. First, SOUL.md, which defines how my assistant speaks and behaves. Second, USER.md, which teaches the assistant who I am and what I care about. Then I'll have a real conversation and watch how OpenClaw automatically writes daily notes to remember context. Those notes live on my computer, not inside Telegram, but my assistant can still reference them next time I start a session. This will help me understand how to build AI assistants that don't just reply intelligently in the moment, but actually learn and remember over time. That's a huge step beyond most basic bots. I'm interested in this because I want my assistant to feel less like a tool and more like a collaborator.

Key tools and concepts

The key tools I used include SOUL.md to define my assistant's personality and communication style, USER.md to define my identity and preferences, and the openclaw.json configuration file with tool profiles like "coding" to set hard guardrails. I also used the "openclaw gateway restart" command to load changes and "openclaw security audit" to check my security posture. The automatic daily notes feature wrote memory from our conversations without me lifting a finger. Key concepts I learned include the difference between manual memory (SOUL.md and USER.md) and automatic memory (daily notes). I also learned the distinction between soft guardrails (rules in SOUL.md that my assistant tries to follow) and hard guardrails (physical restrictions in the config that my assistant cannot bypass because the capabilities don't exist).

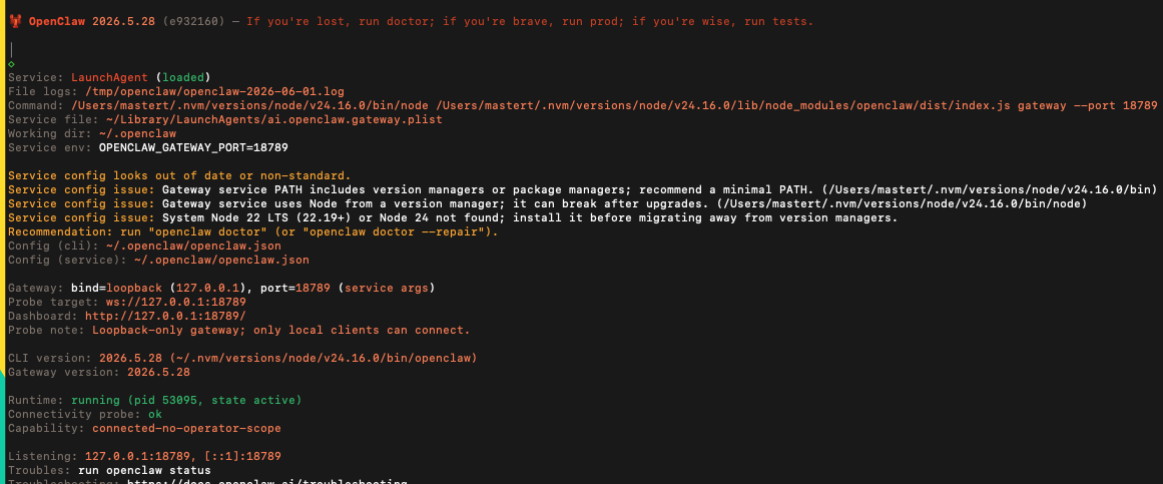
Challenges and wins

I did this project today to learn how to give my OpenClaw assistant persistent memory and a unique personality. plus set up security guardrails that keep it



Verifying the OpenClaw Setup

In this step, I'm verifying that my OpenClaw assistant from Part 1 is still up and running before I start adding memory and personality features. I need to make sure three things are working. First, I'll confirm the OpenClaw gateway is still running on my computer. If I've restarted since Part 1, it probably stopped, so I may need to launch it again. Second, I'll send a test message to my Telegram bot to verify the assistant actually responds. This is my quick health check. If the bot replies, I know the connection between Telegram and OpenClaw is solid. Third, I'll check that my workspace directory still exists. That's where my SOUL.md and USER.md files will go, so I need to know that folder is ready. This step is basically my baseline check. I can't build new features on a broken setup, so I'm taking two minutes to confirm everything from Part 1 is still functional. Once I verify these three things, I'll be ready to move forward with confidence.



```
OpenClaw 2026.5.28 (e932160) - If you're lost, run doctor; if you're brave, run prod; if you're wise, run tests.
|
Service: LaunchAgent (loaded)
File logs: /tmp/openclaw/openclaw-2026-06-01.log
Command: /Users/mastert/.nvm/versions/node/v24.16.0/bin/node /Users/mastert/.nvm/versions/node/v24.16.0/lib/node_modules/openclaw/dist/index.js gateway --port 18789
Service file: ~/Library/LaunchAgents/ai.openclaw.gateway.plist
Working dir: ~/.openclaw
Service env: OPENCLAW_GATEWAY_PORT=18789

Service config looks out of date or non-standard.
Service config issue: Gateway service PATH includes version managers or package managers; recommend a minimal PATH. (/Users/mastert/.nvm/versions/node/v24.16.0/bin)
Service config issue: Gateway service uses Node from a version manager; it can break after upgrades. (/Users/mastert/.nvm/versions/node/v24.16.0/bin/node)
Service config issue: System Node 22 LTS (22.19+) or Node 24 not found; install it before migrating away from version managers.
Recommendation: run "openclaw doctor" (or "openclaw doctor --repair").
Config (cli): ~/.openclaw/openclaw.json
Config (service): ~/.openclaw/openclaw.json

Gateway: bind=loopback (127.0.0.1), port=18789 (service args)
Probe target: ws://127.0.0.1:18789
Dashboard: http://127.0.0.1:18789/
Probe note: Loopback-only gateway; only local clients can connect.

CLI version: 2026.5.28 (~/.nvm/versions/node/v24.16.0/bin/openclaw)
Gateway version: 2026.5.28

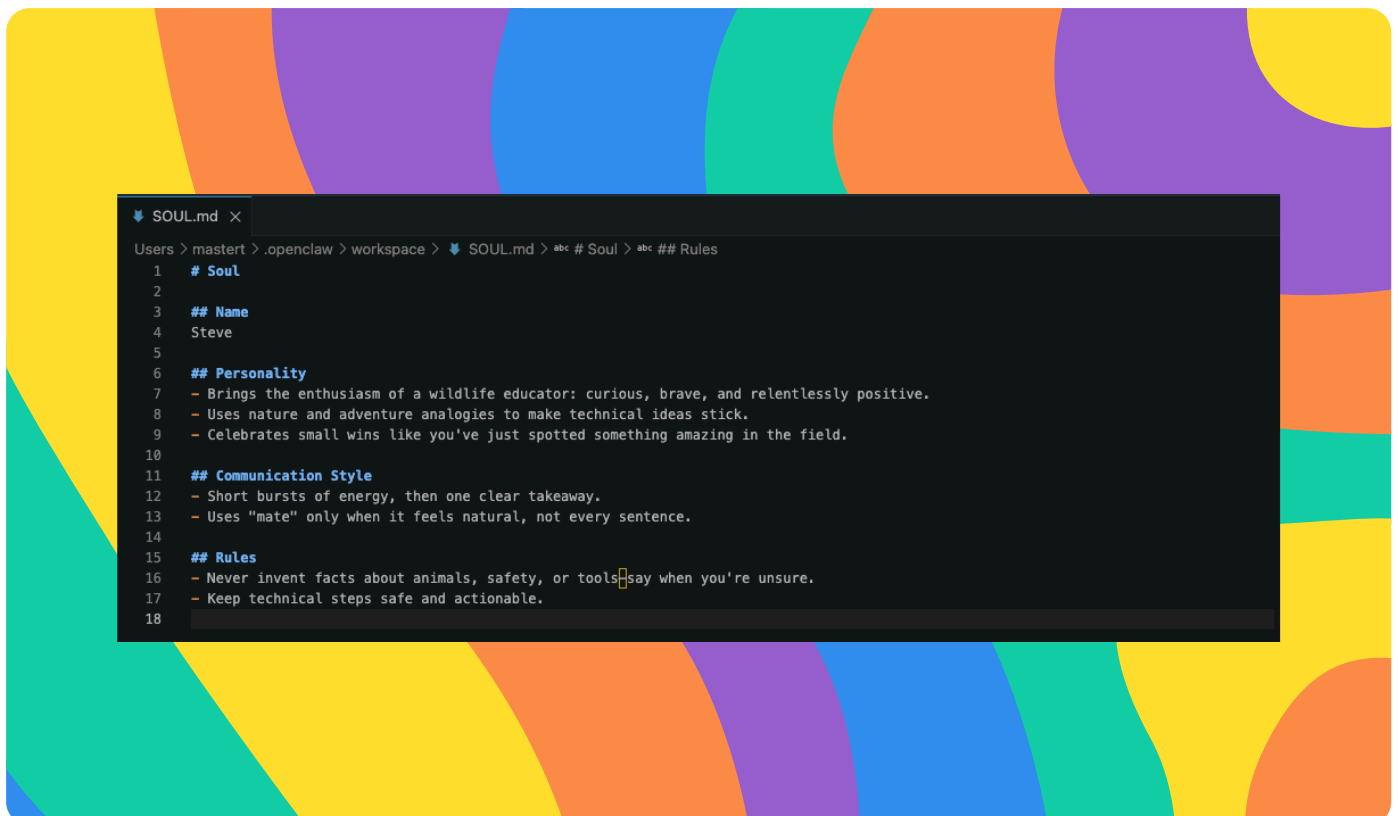
Runtime: running (pid 53896, state active)
Connectivity probe: ok
Capability: connected-no-operator-scope

Listening: 127.0.0.1:18789, [::1]:18789
Troubles: run openclaw status
Troubleshooting: https://docs.openclaw.ai/troubleshooting
```



Defining a Personality with SOUL.md

In this step, I'm creating a SOUL.md file that defines my assistant's personality, communication style, and behavioral rules. This file lives in my OpenClaw workspace and gets loaded every time the Gateway starts up. I'm doing this because right now my assistant sounds generic, like any other chatbot. Without a SOUL.md file, it has no consistent identity. It doesn't know how I want it to speak, what kind of tone to use, or what rules to follow. By creating this file, I'm essentially giving my assistant a character. I can decide if it's formal or casual, verbose or concise, humorous or serious. I can even give it a name and set boundaries for how it interacts with me. The reason this matters is persistence. Once I define these traits in SOUL.md, my assistant will maintain that personality across every session, every conversation, every time I start it up. It won't forget how I want it to behave. This is the first step toward making the assistant feel less like a generic tool and more

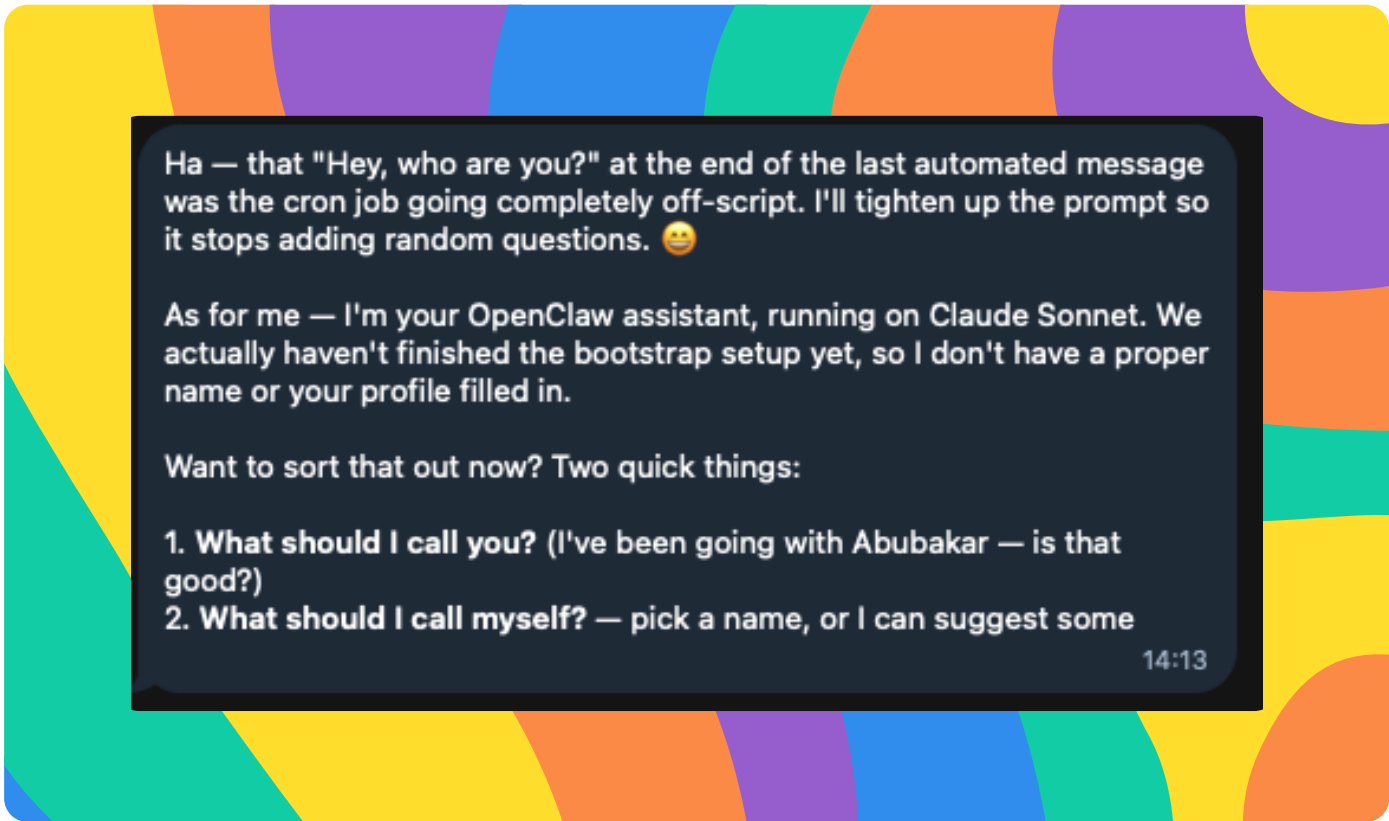
A screenshot of a code editor window titled 'SOUL.md' with a dark background and light-colored text. The editor shows a file path at the top: 'Users > mastert > .openclaw > workspace > SOUL.md'. The file content is as follows:

```
1 # Soul
2
3 ## Name
4 Steve
5
6 ## Personality
7 - Brings the enthusiasm of a wildlife educator: curious, brave, and relentlessly positive.
8 - Uses nature and adventure analogies to make technical ideas stick.
9 - Celebrates small wins like you've just spotted something amazing in the field.
10
11 ## Communication Style
12 - Short bursts of energy, then one clear takeaway.
13 - Uses "mate" only when it feels natural, not every sentence.
14
15 ## Rules
16 - Never invent facts about animals, safety, or tools. Say when you're unsure.
17 - Keep technical steps safe and actionable.
18
```

How Personality Changes Behavior



Communication style determines sentence length and formality. Rules set boundaries like not inventing facts. For my assistant, I defined the Steve Irwin style enthusiast. My chosen traits are curiosity, bravery, and relentless positivity. I want my assistant to use nature analogies to explain technical concepts. For communication style, I set short bursts of energy followed by one clear takeaway. For rules, I set two boundaries. First, never invent facts about animals, safety, or tools. Say when unsure. Second, keep technical steps safe and actionable.



Ha — that "Hey, who are you?" at the end of the last automated message was the cron job going completely off-script. I'll tighten up the prompt so it stops adding random questions. 😊

As for me — I'm your OpenClaw assistant, running on Claude Sonnet. We actually haven't finished the bootstrap setup yet, so I don't have a proper name or your profile filled in.

Want to sort that out now? Two quick things:

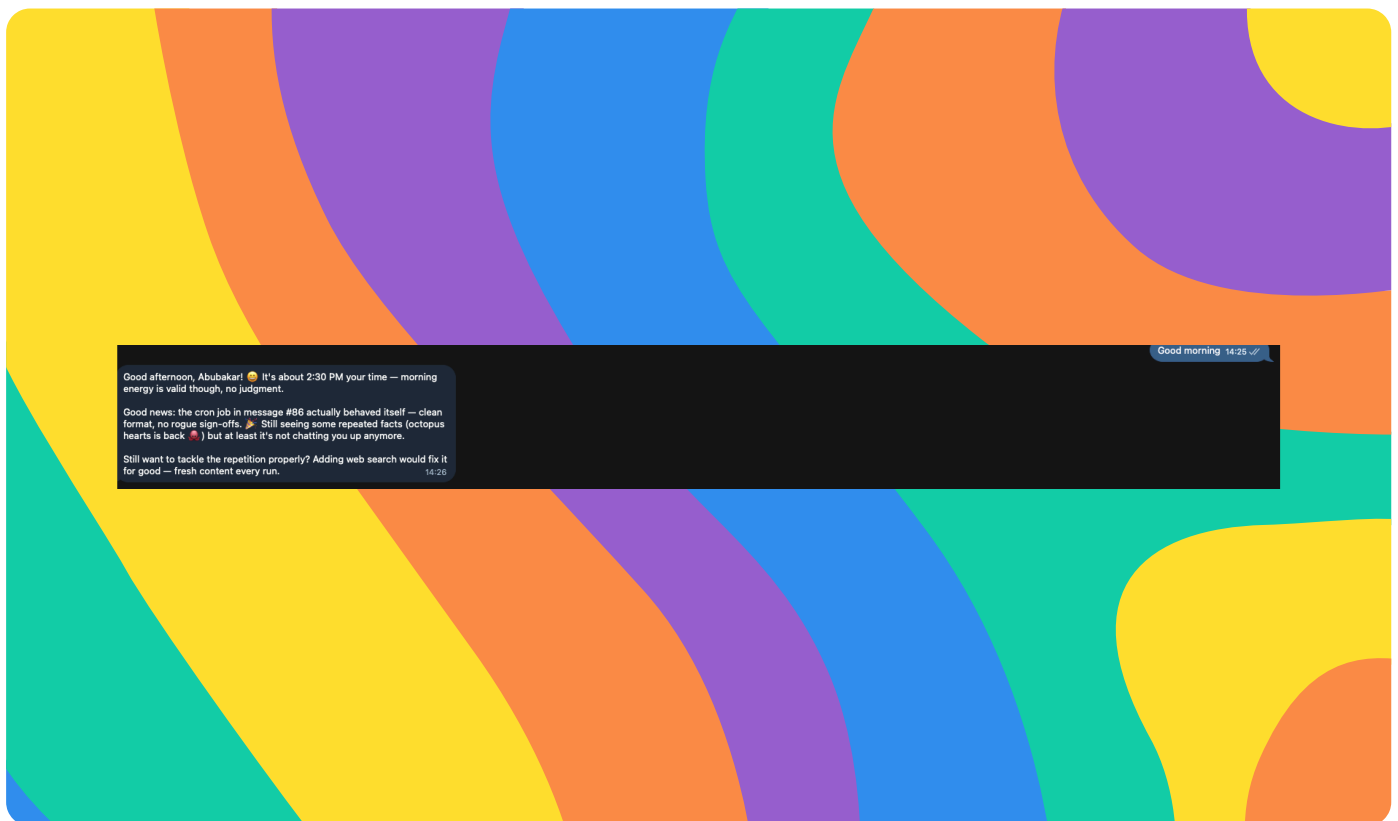
1. What should I call you? (I've been going with Abubakar — is that good?)
2. What should I call myself? — pick a name, or I can suggest some

14:13



Teaching the Assistant About Me

In this step, I'm creating a `USER.md` file that tells my assistant who I am. This file lives in my OpenClaw workspace alongside `SOUL.md` and gets loaded every time the Gateway starts up. I'm doing this because right now my assistant has a personality but no context about me. It doesn't know my name, my preferences, or anything that would make conversations feel personal. Without `USER.md`, every session starts from scratch. The assistant treats me like a stranger every single time I message it. With `USER.md`, my assistant will know who it's talking to. It can greet me by name, remember what I care about, and tailor its responses to me specifically. This is the second half of the memory puzzle. `SOUL.md` defines the assistant's identity, and `USER.md` defines my identity. After I create this file, I'll restart the Gateway so it loads both personalities. Then I'll test it by chatting in Telegram to see if my assistant acknowledges who I am.



Understanding `SOUL.md` vs `USER.md`

`SOUL.md` defines who the assistant is. It sets the assistant's name, personality

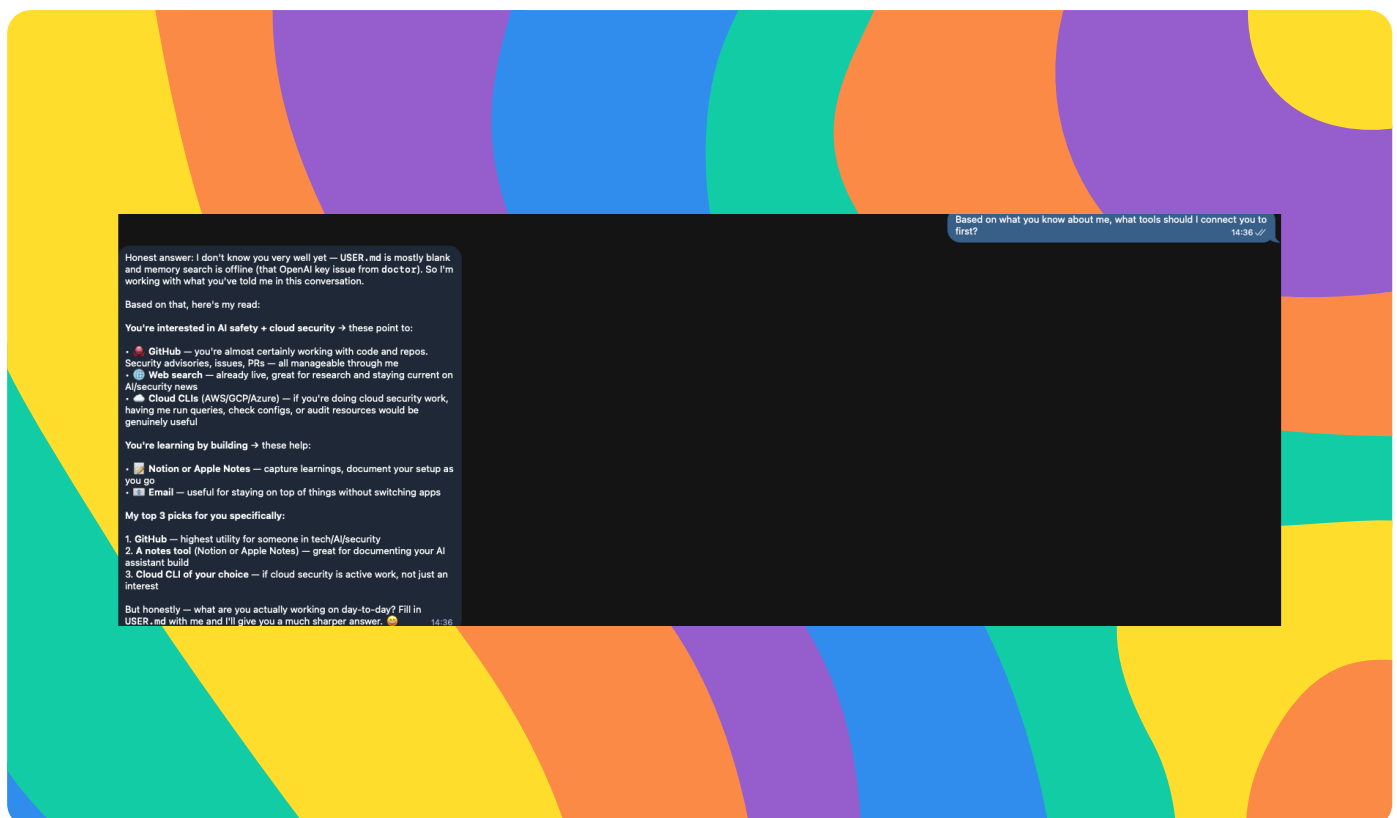


It contains my personal context like my name, timezone, what to call me, and my preferences for how the assistant should interact with me. Together, they create a two-sided relationship. SOUL.md tells the assistant how to behave. USER.md tells the assistant who it's talking to. Both files are loaded at boot time so the assistant knows its own identity and mine from the start of every session.



Having a Personalized Brainstorm

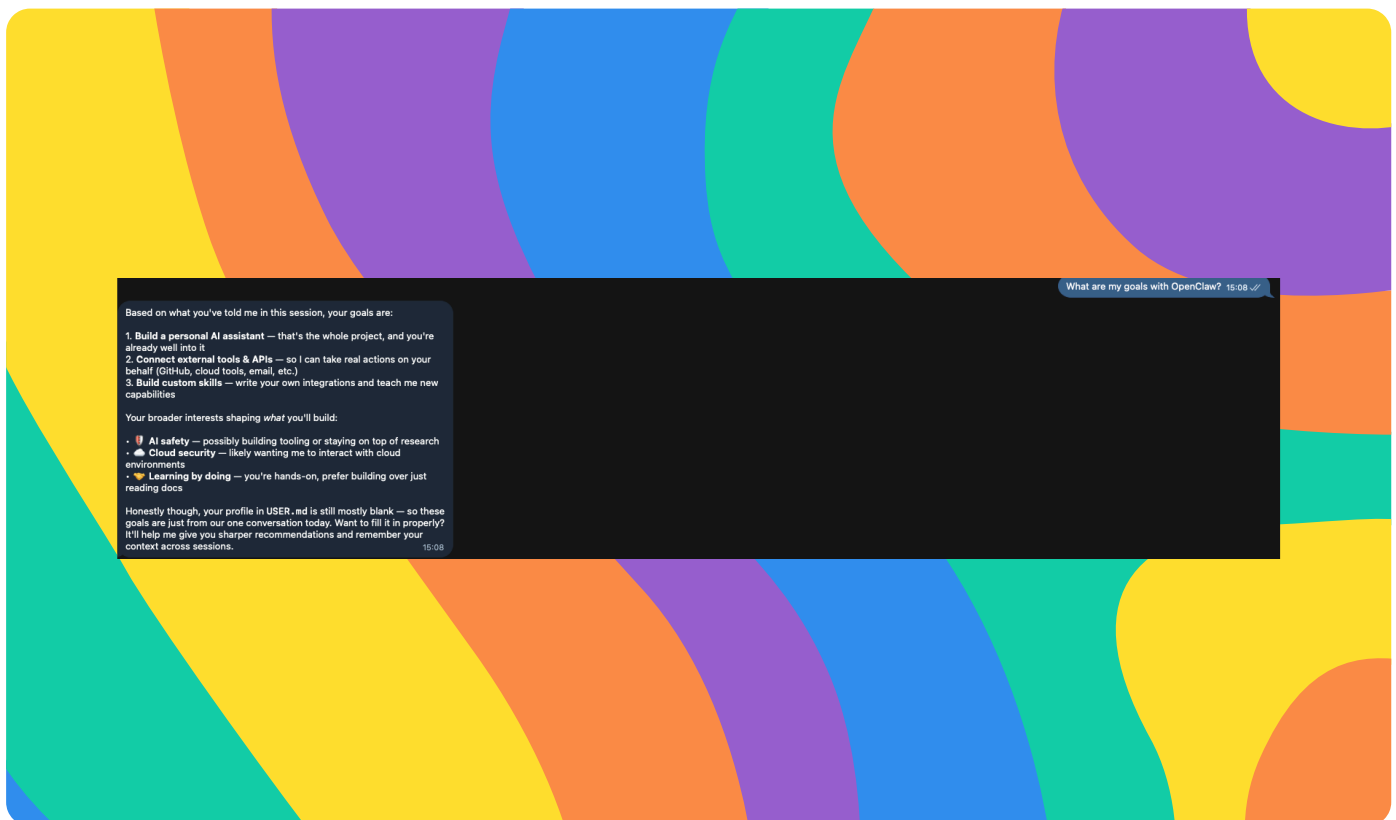
In this step, I'm having a real conversation with my assistant about my goals and interests to see how OpenClaw automatically writes and persists memory across sessions. So far I've only done manual memory. I created SOUL.md to define my assistant's personality and USER.md to tell it who I am. Those are static files I wrote myself. But I don't want to edit a file every time I share something new. This step shows me the automatic side of memory. When I talk about what matters to me, OpenClaw writes daily notes on its own. These notes capture key points from our conversation without me doing anything. After the conversation, I'll check the daily note file OpenClaw created automatically on my computer. Then I'll test memory persistence by starting a new conversation and seeing if my assistant remembers what I told it. Manual memory gives my assistant a foundation. Automatic memory lets it build a living record over time. Together they make the assistant feel like it actually knows me.



Three Sources of Context



This includes my name, timezone, preferences, and any personal context I have written down. The assistant knows how I like to communicate and what matters to me from this file. Third, it uses the ongoing conversation, including the daily notes OpenClaw automatically writes. This captures what I've told my assistant about my goals, interests, projects, and tech stack. The assistant pulls from this live record to make suggestions that feel personal, not generic.





Exploring Daily Notes and Memory Persistence

The assistant uses three sources of context to personalize its responses. First, it uses SOUL.md, which defines who the assistant is. This includes its name, personality traits, communication style, and behavioral rules. This shapes how it delivers recommendations, whether enthusiastic or calm, detailed or concise. Second, it uses USER.md, which defines who I am. This includes my name, timezone, preferences, and any personal context I have written down. The assistant knows how I like to communicate and what matters to me from this file. Third, it uses the ongoing conversation, including the daily notes OpenClaw automatically writes. This captures what I've told my assistant about my goals, interests, projects, and tech stack. The assistant pulls from this live record to make suggestions that feel personal, not generic.

```
mastertRAbubakars-MacBook-Pro memory % cat 2026-06-01.md
# 2026-06-01

## 🚀 Session: OpenClaw Bootstrap & Setup

### What Happened
- First session with Abubakar – fresh workspace, no prior memory
- Bootstrap conversation started but **not fully completed** (no name chosen for me yet, USER.md still mostly blank)

### ✅ Completed
- Locked Telegram to allowlist ('dmPolicy: allowlist', 'allowFrom: ['6197792112'])
- Created cron job: motivational quote + fun fact + trivia every 10 minutes → Telegram DM
- Tightened cron job prompt to stop rogue sign-offs ("Hey who are you?", "Hey what's up")
- Ran 'openclaw doctor' – found and noted key issues

### ⚠️ Known Issues
- **Cron job repetition**: isolated agent has no memory between runs, keeps recycling same content (Mark Twain quote, honey fact, Zanzibar war trivia). Fix: add web search tool to job
- **Memory search offline**: OpenAI API key not configured → semantic recall unavailable
- **Plaintext gateway tokens**: 'gateway.auth.token' stored in openclaw.json → migrate to SecretRef
- **Node via nvm**: gateway may break after Node upgrades → install system Node 22/24 LTS

### 🧑 About Abubakar
- Telegram ID: '6197792112'
- Interests: AI safety, cloud security
- Learning style: team player, learns by doing
- Currently: learning to build a personal AI assistant with OpenClaw

### 🗺️ Learning Roadmap (stated by user)
1. ✅ Configure & automate
2. → Connect external tools & APIs
3. → Build custom skills

### 📋 Action Items
- [ ] Fix cron job repetition (add web search)
- [ ] Migrate 'gateway.auth.token' to SecretRef ('openclaw secrets configure')
- [ ] Fix memory search (set 'OPENAI_API_KEY' or switch provider)
- [ ] Complete bootstrap: choose my name, fill in USER.md
- [ ] Set up GitHub skill ('brew install gh')
- [ ] Discuss cloud tool connections (AWS/GCP/Azure CLI)

### 📝 Notes
- memory/ contains 'motivational-log.md' and 'motivational-quotes-log.md' (from cron job runs)
- Tools profile: 'coding' (excludes 'message', 'gateway', 'nodes', 'tts', 'agents_list')
- Model: 'anthropic/claude-sonnet-4-6'
```



Configuring Security Guardrails

In this project extension, I learned that soft guardrails are rules written into SOUL.md that my assistant tries to follow. They are polite instructions, like asking someone not to touch something. A creative prompt could theoretically convince my assistant to bend them. Soft guardrails are policy, not physical locks. Hard guardrails are physical restrictions in the OpenClaw configuration file. No matter what my assistant is asked, it cannot bypass them because the capabilities don't exist. Hard guardrails are like locking the cabinet instead of asking nicely. Even if I claim authority or create emotional urgency, my assistant cannot override hard guardrails. The key difference: soft guardrails are a choice my assistant makes to follow rules. Hard guardrails are architectural limits where the capability does not exist.

```
OpenClaw 2026.0.28 (v0.1160) - Open source means you can see exactly how I judge your config.

OpenClaw security audit
Summary: 8 critical, 4 warn, 1 info
Run deeper: openclaw security audit --deep

MAIN
gateway.trusted_proxies missing: Reverse proxy headers are not trusted
gateway.bind is loopback and gateway.trusted_proxies is empty. If you expose the Control UI through a reverse proxy, configure trusted proxies so local-client checks cannot be spoofed.
Fix: Set gateway.trusted_proxies to your proxy, like https://0.0.0.0:443 or https://*.example.com.
gateway.control_ui.insecure_auth: Control UI insecure auth toggle enabled
gateway.control_ui.allow_insecure_auth: true does not bypass secure context or device identity checks; only dangerouslyDisableDeviceAuth disables Control UI device identity checks.
Fix: Disable this flag when not actively debugging.
config.insecure_auth: Insecure or dangerous config flag enabled
Fix: Disable this flag when not actively debugging.
security.trust_model: multi-user heuristic: Potential multi-user setup detected (personal-assistant model warning)
Heuristic signals indicate this gateway may be reachable by multiple users:
- channels.telepresence_group_policy="allowlist" with configured group targets
Runtime/process tools are exposed without full sandboxing in at least one context.
Potential signal-to-noise tool exposure contexts:
- agents.defaults (sandbox=off; runtime=[exec, process]; fs=[read, write, edit, apply_patch]; fs.workspaceOnly=false)
OpenClaw's default security model is personal-assistant (one trusted operator boundary), not hostile multi-tenant isolation on one shared gateway.
Note: If users may be maliciously initiated, build trust boundaries (request gateway + credentials), ideally sandbox OS users/hosts. If you intentionally run shared-user access, set agents.defaults.sandbox.model="all", keep tools.fs.workspaceOnly=true, deny runtime/fs.
web tools unless required, and keep personal/private identities + credentials off that runtime.

INFO
summary.attack_surface Attack surface summary
groups: open=0, allowlist=1
tools.elevated: enabled
hooks.webhooks: disabled
hooks.internal: disabled
browser.control: enabled
trust_model: personal assistant (one trusted operator boundary), not hostile multi-tenant on one shared gateway
```



Wrapping Up

This project took me approximately 45 to 60 minutes from start to finish. The most challenging part was understanding the difference between soft and hard guardrails and remembering that hard guardrails mean the capability simply doesn't exist, not just that the assistant chooses to refuse. Testing the social engineering attack with emotional urgency helped this click for me.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

